

Implement Client Script & UI Policy (Incident)

Sample project documentation

ServiceNow Incident Form Customization using Client Scripts and UI Policies

Project Description:

This project improves the ServiceNow Incident form by adding client-side logic that guides users while they log and resolve incidents. Without validation, the form lets users save incomplete or inconsistent data: short descriptions are vague, resolution details are missing, and every field is shown to every user regardless of the incident state. This lowers data quality and slows down resolution.

The solution uses two ServiceNow client-side features. **UI Policies** dynamically make fields mandatory, read-only or visible based on conditions such as State, Category and Priority. **Client Scripts** (onLoad, onChange and onSubmit) run in the browser to show helpful messages, validate input and stop invalid submissions before they reach the server.

Built and tested on a ServiceNow Personal Developer Instance (PDI) and packaged as an Update Set, the project gives service desk agents a cleaner, guided form, reduces data-entry errors, and makes incidents easier to route and resolve.

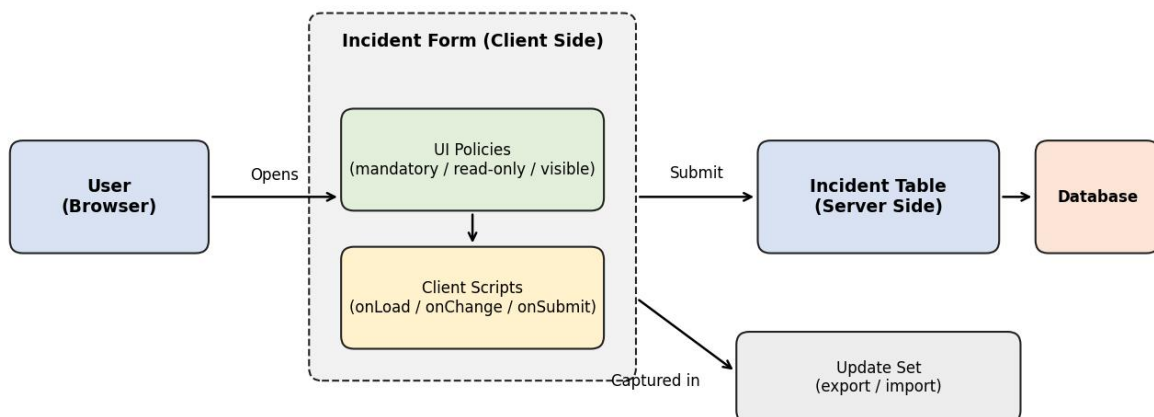
Scenarios

Scenario 1: An agent moves an incident to the Resolved state. A UI Policy immediately makes Resolution code and Resolution notes visible and mandatory, so the incident cannot be resolved without documenting how it was fixed.

Scenario 2: A caller reports a critical outage and the agent sets Priority to 1 - Critical. The form makes Assignment group mandatory, and a Client Script shows an info message reminding the agent to route the incident to the right team immediately.

Scenario 3: An agent enters a very short Short description such as "help" and clicks Submit. The onSubmit Client Script blocks the save and shows an error message asking for a more descriptive summary of at least 10 characters.

Technical Architecture:



Description: The project follows ServiceNow's client-server model. When a user opens the Incident form in the browser, the platform loads the UI Policies and Client Scripts defined for the Incident table. UI Policies control field behavior (mandatory, read-only, visible) as conditions change, while Client Scripts handle form load, field changes and form submission. Only data that passes these client-side checks is submitted to the Incident table on the server and stored in the ServiceNow database. All configuration is captured in an Update Set so it can be moved between instances.

Data model: This project uses the existing Incident table (incident, extended from the Task table) and does not create a new table. The key relationships used are shown below.

Incident field	Reference table	Purpose in this project
caller_id	User (sys_user)	Person reporting the incident
assignment_group	Group (sys_user_group)	Made mandatory for Priority 1 - Critical
assigned_to	User (sys_user)	Agent working on the incident
cmdb_ci	Configuration Item (cmdb_ci)	Affected item, mandatory for hardware incidents

Pre-requisites:

- **ServiceNow Developer Account & PDI:** [ServiceNow Developer Program](#)
- **ServiceNow Product Documentation:** [ServiceNow Docs](#)
- **Incident Management Basics:** Understanding of incident fields, states and priority.
- **Client Scripts & UI Policies:** Knowledge of when to use each, and of the g_form API (see ServiceNow Docs).
- **JavaScript Fundamentals:** Needed to write onLoad, onChange and onSubmit scripts.
- **Update Sets:** Understanding of how to capture, export and import configuration changes.
- **Browser Developer Tools:** Useful for debugging client scripts (Chrome / Edge).

Project Workflow:

Milestone 1: Instance Setup and Requirement Analysis

- **Activity 1.1:** Sign up for the ServiceNow Developer Program and request a Personal Developer Instance (PDI).
- **Activity 1.2:** Analyse the requirements and identify the Incident fields that need validation or dynamic behavior.
- **Activity 1.3:** Design the solution by deciding which rules belong in UI Policies and which need Client Scripts.
- **Activity 1.4:** Create a dedicated Update Set to capture all changes.

Milestone 2: UI Policy Development

- **Activity 2.1:** Create UI Policies that control mandatory fields based on State and Priority.
- **Activity 2.2:** Create UI Policies that show, hide or lock fields based on Category and State.

Milestone 3: Client Script Development

- **Activity 3.1:** Write an onLoad Client Script to show guidance when the form opens.
- **Activity 3.2:** Write an onChange Client Script to react when Category or Priority changes.
- **Activity 3.3:** Write an onSubmit Client Script to validate the form before saving.

Milestone 4: Testing and Validation

- **Activity 4.1:** Prepare test cases for every UI Policy and Client Script.
- **Activity 4.2:** Execute the tests and record the results with screenshots.

Milestone 5: Deployment

- **Activity 5.1:** Export the Update Set from the development instance.
- **Activity 5.2:** Import, preview and commit the Update Set on the target instance.

Milestone 6: Conclusion

Milestone 1: Instance Setup and Requirement Analysis

In this milestone, we set up the ServiceNow environment and define exactly what the Incident form needs to enforce, so that the right feature (UI Policy or Client Script) is chosen for each rule.

Activity 1.1: Get a ServiceNow Personal Developer Instance (PDI)

- **Step 1 — Create an account:** Visit <https://developer.servicenow.com> and sign up for a free developer account.
- **Step 2 — Request an instance:** After logging in, click Request Instance and choose the latest release.
- **Step 3 — Open the instance:** Once the instance is ready, note the instance URL and admin credentials, and log in as admin.

[Insert screenshot: PDI home page / instance details]

Activity 1.2: Analyse the Requirements

The following rules were identified for the Incident form:

#	Requirement	Mechanism	Trigger
1	Resolution code and notes mandatory when resolving	UI Policy	State is Resolved
2	Assignment group mandatory for critical incidents	UI Policy	Priority is 1 - Critical
3	Configuration item mandatory for hardware issues	UI Policy	Category is Hardware
4	Guidance message when the form opens	onLoad Client Script	Form load
5	Hint message when Category changes	onChange Client Script	Category changes
6	Block saving with a very short Short description	onSubmit Client Script	Form submit

Activity 1.3: Define the Solution Design

- **Use UI Policies** for simple, condition-based field behavior (mandatory, read-only, visible), as they can be configured without code.
- **Use Client Scripts** for logic that needs scripting: messages, custom validation and blocking a submit.
- **Set UI Type to All** so the behavior works in both the platform UI and Workspace.

Activity 1.4: Create an Update Set

- Navigate to System Update Sets > Local Update Sets and click New.
- Name it "Incident Client Script and UI Policy" and set it as the current update set.
- All UI Policies and Client Scripts created next are automatically captured in this Update Set.

[Insert screenshot: Update Set created and set as current]

Milestone 2: UI Policy Development

In this milestone, we create UI Policies on the Incident table to control field behavior without writing server code.

Activity 2.1: Create UI Policies for Mandatory Fields

Open the form: Navigate to System UI > UI Policies and click New. Set the Table to Incident, then fill in the details for each policy below.

UI Policy	Condition	UI Policy Action
Resolution details required	State is Resolved	Resolution code and Resolution notes: Mandatory = True, Visible = True
Critical needs a group	Priority is 1 - Critical	Assignment group: Mandatory = True
Hardware needs a CI	Category is Hardware	Configuration item: Mandatory = True

- Leave the Global checkbox unchecked so the policy applies only to the Incident table.
- Tick **Reverse if false** so fields return to normal when the condition no longer applies.
- On the Script tab, leave Run scripts unchecked, as these policies need no code.

[Insert screenshot: UI Policy form with condition and actions]

Activity 2.2: Create UI Policies to Show, Hide and Lock Fields

- **Lock resolved incidents:** Condition State is Closed; add UI Policy Actions that set key fields (such as Short description) to Read only.
- **Show details when needed:** Show Hold reason only when State is On Hold, and hide it otherwise.
- For each UI Policy, set the UI Type to All and click Submit.

[Insert screenshot: UI Policy list for the Incident table]

Milestone 3: Client Script Development

In this milestone, we write three Client Scripts for the Incident table, one for each script type. Navigate to System Definition > Client Scripts, click New, and set the Table to Incident.

Activity 3.1: onLoad Client Script

Name: Incident - Show guidance on load | **Type:** onLoad. It displays a message when the form opens and sets a default contact type for new records.

```
function onLoad() {
    g_form.addInfoMessage('Please provide complete details to help resolve this incident faster.');
```



```
    if (g_form.isNewRecord() && g_form.getValue('contact_type') == '') {
        g_form.setValue('contact_type', 'phone');
    }
}
```

Activity 3.2: onChange Client Script

Name: Incident - Category change hint | **Type:** onChange | **Field name:** Category. It guides the user when a category is picked.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    g_form.hideFieldMsg('category', true);

    if (newValue == 'software') {
        g_form.showFieldMsg('category',
            'Mention the application name and version in the Short description.', 'info');
    } else if (newValue == 'hardware') {
        g_form.showFieldMsg('category',
            'Please select the affected Configuration item.', 'info');
    }
}
```

Activity 3.3: onSubmit Client Script

Name: Incident - Validate before submit | **Type:** onSubmit. It stops the save when the data is not good enough.

```
function onSubmit() {
    var shortDesc = g_form.getValue('short_description').trim();

    if (shortDesc.length < 10) {
        g_form.showFieldMsg('short_description',
            'Short description must be at least 10 characters.', 'error');
        return false; // stops the submit
    }
    return true;
}
```

- Set **UI Type** to All and tick **Active** on each script before saving.
- The field values used above (for example software, hardware and phone) are the backend values; confirm them on your instance.

[Insert screenshot: Client Script record and the message shown on the form]

Milestone 4: Testing and Validation

In this milestone, we verify that every UI Policy and Client Script behaves as expected on the Incident form.

Activity 4.1: Test Cases

#	Test Case	Expected Result	Status
1	Set State to Resolved and save without resolution notes	Resolution code and notes show as mandatory; save is blocked	Pass / Fail
2	Set Priority to 1 - Critical and leave Assignment group empty	Assignment group becomes mandatory	Pass / Fail
3	Select Category = Hardware	Configuration item becomes mandatory and a hint appears	Pass / Fail
4	Open a new Incident	Guidance message is displayed; contact type defaults to Phone	Pass / Fail
5	Change Category to Software	Info message appears below the Category field	Pass / Fail
6	Enter 'help' in Short description and submit	Error message appears and the record is not saved	Pass / Fail
7	Switch State back from Resolved	Resolution fields are no longer mandatory (Reverse if false)	Pass / Fail

Activity 4.2: Execute and Record Results

- Create a new incident from Incident > Create New and run each test case.
- Record the result of each test and capture screenshots of the messages and mandatory-field markers.
- Fix any failing scenario by correcting the condition or script, then retest.

[Insert screenshot: Incident form showing a mandatory field and an error message]

Milestone 5: Deployment

In this milestone, we package the work as an Update Set and move it to another instance, so the solution can be reused or reviewed.

Activity 5.1: Export the Update Set

- Go to System Update Sets > Local Update Sets and open "Incident Client Script and UI Policy".
- Review the Customer Updates list to confirm all UI Policies, UI Policy Actions and Client Scripts are included.
- Set State to Complete and use Export to XML to download the file.

Activity 5.2: Import and Commit on the Target Instance

- On the target instance, go to System Update Sets > Retrieved Update Sets and choose Import Update Set from XML.
- Upload the XML file, open the retrieved Update Set and click **Preview Update Set**.

- Resolve any conflicts, then click **Commit Update Set**.
- Open an Incident record and repeat the key test cases to confirm the behavior.

[Insert screenshot: Update Set committed successfully]

Important Notes:

- Client Scripts and UI Policies run only in the browser, so they do not apply to records created through import sets, REST APIs or background scripts. Use Business Rules or Data Policies for server-side enforcement.
- Keep the UI Type set to All so the behavior also works in Workspace.
- Avoid duplicating logic: if a rule can be done with a UI Policy, prefer it over a Client Script.

Exploring the Incident Form:

Create New Incident Page:

[Insert screenshot: Incident form on load with guidance message]

Description: The standard Incident form opens with the onLoad guidance banner at the top. New records default the contact type to Phone, and fields such as Resolution code and Resolution notes stay hidden until the incident is resolved.

Validation Section:

[Insert screenshot: Mandatory fields and error message on the Incident form]

Description: When a user sets Priority to 1 - Critical, Assignment group is marked mandatory. When State changes to Resolved, Resolution code and Resolution notes appear as mandatory. If the Short description is too short, the onSubmit script displays an error under the field and blocks the save.

Conclusion:

This project shows how Client Scripts and UI Policies on the ServiceNow Incident table improve data quality and the agent experience without any server-side changes. UI Policies handle condition-based field behavior, while onLoad, onChange and onSubmit Client Scripts add guidance and validation directly in the browser. The work was configured and tested on a Personal Developer Instance and packaged as an Update Set for easy deployment. Looking ahead, the solution can be extended with Business Rules or Data Policies for server-side enforcement, Catalog Client Scripts for the Service Portal, and additional UI Policies for other tables such as Problem and Change.